```
"""
nexus.core.config
==================

Typed, layered configuration for
NEXUS.

Deliberately implemented with the
standard library only
(``dataclasses`` +
``os.environ``) rather than pydantic-
settings, so the core package has
zero
third-party dependencies and can be
imported/tested anywhere Python runs,
including inside minimal automation
containers. Higher layers (e.g. the
FastAPI-based developer API
introduced in a later module) are
free to use
pydantic for *their own*
request/response schemas — that is a
separate
concern from this foundational config
loader.
```

```
Configuration precedence (highest
wins):
    1. Explicit keyword arguments
passed to ``load_config()``.
    2. Real process environment
variables (``os.environ``).
    3. Values from a ``.env`` file,
if present.
    4. Hard-coded defaults defined on
the dataclasses below.

No secret values are ever logged.
``NexusConfig.redacted()`` produces a
copy suitable for logging/debugging
with API keys masked.
"""

from __future__ import annotations

import os
from dataclasses import dataclass,
field, fields
from enum import Enum
from pathlib import Path
from typing import Any

from nexus.core.exceptions import
ConfigurationError
```

```
_SECRET_FIELD_KEYWORDS = ("api_key",
"secret", "token", "password", "dsn")


class Environment(str, Enum):
    """Deployment environment.
Controls logging verbosity, guardrail
strictness, etc."""

    DEVELOPMENT = "development"
    STAGING = "staging"
    PRODUCTION = "production"
    TEST = "test"


def _default_instance(cls: type) ->
Any:
    """
    Return a plain default-
constructed instance of a slots
dataclass.

    Needed because with
``@dataclass(slots=True)``, accessing
a field via
    the *class* (e.g.
``_voice_defaults.wake_word``)
returns a slot member
    descriptor object, not the
```

```
field's default value — unlike
ordinary
    (non-slotted) dataclasses or
plain classes. Constructing a
throwaway
    instance and reading the
attribute off of *that* gives the
real default.
    """
    return cls()


def _parse_dotenv(path: Path) ->
dict[str, str]:
    """
    Minimal ``.env`` parser
(KEY=VALUE per line, '#' comments, no
interpolation).

    Intentionally does not support
the full dotenv spec (multiline
values,
    variable expansion) — NEXUS's
needs are simple key/value pairs, and
a
    minimal parser means one less
third-party dependency in the core
layer.
    """
```

```
    values: dict[str, str] = {}
    if not path.exists():
        return values

    for lineno, raw_line in
enumerate(path.read_text(encoding="ut
f-8").splitlines(), start=1):
        line = raw_line.strip()
        if not line or
line.startswith("#"):
            continue
        if "=" not in line:
            raise ConfigurationError(
                f"Malformed line in
{path}:{lineno} — expected
KEY=VALUE",
                details={"file":
str(path), "line": lineno},
            )
        key, _, value =
line.partition("=")
        key = key.strip()
        value =
value.strip().strip('"').strip("'")
        values[key] = value
    return values


@dataclass(frozen=True, slots=True)
```

```
class VoiceConfig:
    """Configuration for the voice
pipeline (wake word, VAD, STT,
TTS)."""

    wake_word: str = "nexus"
    wake_word_sensitivity: float =
0.55
    stt_model: str = "whisper-large-
v3-turbo"
    stt_streaming: bool = True
    tts_provider: str = "elevenlabs"
# or "xtts_v2" for fully local
    tts_voice_id: str = ""
    sample_rate_hz: int = 16_000
    vad_aggressiveness: float = 0.5
# Silero VAD threshold, 0.0-1.0


@dataclass(frozen=True, slots=True)
class LLMConfig:
    """
    Configuration for the
interchangeable LLM backend.

    ``provider`` selects which
registered ``LLMProvider``
implementation the
    DI container resolves (see
```

```
``nexus.core.interfaces.LLMProvider`` and the
    concrete adapters introduced in the "Brain" module). Keeping provider
    selection as a plain string (rather than importing every SDK here) is
    what keeps this file dependency-free.
    """

    provider: str = "claude"  # "claude" | "gpt" | "gemini" | "ollama"
    model_name: str = "claude-sonnet-4-6"
    api_key: str = ""
    base_url: str = ""  # used for local Ollama or self-hosted endpoints
    max_tokens: int = 2048
    temperature: float = 0.4
    request_timeout_s: float = 30.0


@dataclass(frozen=True, slots=True)
class MemoryConfig:
    """Configuration for short/long-term memory backends."""
```

```
    vector_db_provider: str =
"qdrant"  # "qdrant" | "weaviate"
    vector_db_url: str =
"http://localhost:6333"
    postgres_dsn: str = ""
    knowledge_graph_enabled: bool =
False
    neo4j_uri: str = ""
    short_term_max_turns: int = 40
    summary_every_n_turns: int = 20


@dataclass(frozen=True, slots=True)
class SecurityConfig:
    """
    Configuration for the
deterministic guardrail layer.

    ``blocked_package_substrings``
and ``confirm_required_actions`` are
the
    default policy; they are
intentionally data (not code) so an
operator
    can tighten policy via config
without touching guardrail logic
itself.
    """
```

```
    blocked_package_substrings:
tuple[str, ...] = (

"com.google.android.apps.wallet",
        "com.paypal",
        "com.chase",
        "com.bankofamerica",
        "com.venmo",
        "com.squareup.cash",
        "com.coinbase",
    )
    confirm_required_actions:
tuple[str, ...] = (
        "send_message",
        "make_call",
        "delete_app",
        "power_state",
        "delete_file",
        "deploy_code",
    )
    audit_log_path: str =
"./data/audit/audit.log"
    encrypt_memory_at_rest: bool =
True
    max_actions_per_minute: int = 20


@dataclass(frozen=True, slots=True)
class NexusConfig:
```

```
    """
    Root configuration object.
Immutable once constructed
(``frozen=True``)
    so nothing downstream can mutate
shared config out from under another
    component at runtime — any change
requires constructing a new instance.
    """

    environment: Environment =
Environment.DEVELOPMENT
    log_level: str = "INFO"
    log_dir: str = "./data/logs"
    data_dir: str = "./data"
    supported_languages: tuple[str,
...] = ("en", "ne")
    voice: VoiceConfig =
field(default_factory=VoiceConfig)
    llm: LLMConfig =
field(default_factory=LLMConfig)
    memory: MemoryConfig =
field(default_factory=MemoryConfig)
    security: SecurityConfig =
field(default_factory=SecurityConfig)

    def redacted(self) -> dict[str,
Any]:
        """
```

```
        Return a dict representation
safe to log: any field whose name
ends
        in a known secret suffix
(``_api_key``, ``_secret``,
``_token``,
        ``_password``) is masked, at
any nesting depth.
        """

        def _redact_dataclass(obj:
Any) -> Any:
            if hasattr(obj,
"__dataclass_fields__"):
                result: dict[str,
Any] = {}
                for f in fields(obj):
                    value =
getattr(obj, f.name)
                    is_secret =
any(kw in f.name.lower() for kw in
_SECRET_FIELD_KEYWORDS)
                    if is_secret and
value:

result[f.name] = "***REDACTED***"
                    else:

result[f.name] =
```

```
_redact_dataclass(value)
                return result
            if isinstance(obj, Enum):
                return obj.value
            if isinstance(obj,
tuple):
                return list(obj)
            return obj

        return
_redact_dataclass(self)


def load_config(
    *,
    env_file: str | Path = ".env",
    environment_override: Environment
| None = None,
    overrides: dict[str, Any] | None
= None,
) -> NexusConfig:
    """
    Build a ``NexusConfig`` following
the precedence documented in the
    module docstring: kwargs
overrides > os.environ > .env file >
defaults.

    Args:
```

```
        env_file: Path to an optional
``.env`` file. Missing file is not an
            error — it simply
contributes no values.
        environment_override: Force a
specific ``Environment`` regardless
of
            the ``NEXUS_ENVIRONMENT``
variable. Primarily used by tests.
        overrides: Flat dict of top-
level or dotted keys (e.g.
            ``{"llm.model_name":
"gpt-5"}``) that take precedence over
            everything else. Intended
for programmatic construction (e.g.
            in test fixtures), not
for end-user configuration.

    Raises:
        ConfigurationError: if a
required value is missing/invalid, or
the
            ``.env`` file is
malformed.
    """
    dotenv_values =
_parse_dotenv(Path(env_file))

    # See _default_instance()
```

```
docstring: with slots=True
dataclasses we must
    # read defaults off a constructed
instance, not the class itself.
    _voice_defaults =
_default_instance(VoiceConfig)
    _llm_defaults =
_default_instance(LLMConfig)
    _memory_defaults =
_default_instance(MemoryConfig)
    _security_defaults =
_default_instance(SecurityConfig)

    def _get(key: str, default: str)
-> str:
        # Precedence: real
environment variables beat the .env
file.
        if key in os.environ:
            return os.environ[key]
        if key in dotenv_values:
            return dotenv_values[key]
        return default

    def _get_bool(key: str, default:
bool) -> bool:
        raw = _get(key, str(default))
        return raw.strip().lower() in
("1", "true", "yes", "on")
```

```
    def _get_float(key: str, default:
float) -> float:
        raw = _get(key, str(default))
        try:
            return float(raw)
        except ValueError as exc:
            raise ConfigurationError(
                f"Environment
variable {key!r} must be a float, got
{raw!r}"
            ) from exc

    def _get_int(key: str, default:
int) -> int:
        raw = _get(key, str(default))
        try:
            return int(raw)
        except ValueError as exc:
            raise ConfigurationError(
                f"Environment
variable {key!r} must be an int, got
{raw!r}"
            ) from exc

    env_name = (
        environment_override.value
        if environment_override is
not None
```

```
        else
_get("NEXUS_ENVIRONMENT",
Environment.DEVELOPMENT.value)
    )
    try:
        environment =
Environment(env_name)
    except ValueError as exc:
        valid = ", ".join(e.value for
e in Environment)
        raise ConfigurationError(
            f"Invalid
NEXUS_ENVIRONMENT={env_name!r}. Must
be one of: {valid}"
        ) from exc

    voice = VoiceConfig(

wake_word=_get("NEXUS_WAKE_WORD",
_voice_defaults.wake_word),

wake_word_sensitivity=_get_float(

"NEXUS_WAKE_WORD_SENSITIVITY",
_voice_defaults.wake_word_sensitivity
        ),

stt_model=_get("NEXUS_STT_MODEL",
_voice_defaults.stt_model),
```

```
stt_streaming=_get_bool("NEXUS_STT_ST
REAMING",
_voice_defaults.stt_streaming),

tts_provider=_get("NEXUS_TTS_PROVIDER
", _voice_defaults.tts_provider),

tts_voice_id=_get("NEXUS_TTS_VOICE_ID
", _voice_defaults.tts_voice_id),

sample_rate_hz=_get_int("NEXUS_SAMPLE
_RATE_HZ",
_voice_defaults.sample_rate_hz),

vad_aggressiveness=_get_float(

"NEXUS_VAD_AGGRESSIVENESS",
_voice_defaults.vad_aggressiveness
        ),
    )

    llm = LLMConfig(

provider=_get("NEXUS_LLM_PROVIDER",
_llm_defaults.provider),

model_name=_get("NEXUS_LLM_MODEL",
_llm_defaults.model_name),
```

```
api_key=_get("NEXUS_LLM_API_KEY",
_llm_defaults.api_key),

base_url=_get("NEXUS_LLM_BASE_URL",
_llm_defaults.base_url),

max_tokens=_get_int("NEXUS_LLM_MAX_TO
KENS", _llm_defaults.max_tokens),

temperature=_get_float("NEXUS_LLM_TEM
PERATURE",
_llm_defaults.temperature),
        request_timeout_s=_get_float(
            "NEXUS_LLM_TIMEOUT_S",
_llm_defaults.request_timeout_s
        ),
    )

    memory = MemoryConfig(
        vector_db_provider=_get(

"NEXUS_VECTOR_DB_PROVIDER",
_memory_defaults.vector_db_provider
        ),

vector_db_url=_get("NEXUS_VECTOR_DB_U
RL", _memory_defaults.vector_db_url),
```

```
postgres_dsn=_get("NEXUS_POSTGRES_DSN
", _memory_defaults.postgres_dsn),

knowledge_graph_enabled=_get_bool(
            "NEXUS_KG_ENABLED",
_memory_defaults.knowledge_graph_enab
led
        ),

neo4j_uri=_get("NEXUS_NEO4J_URI",
_memory_defaults.neo4j_uri),

short_term_max_turns=_get_int(

"NEXUS_SHORT_TERM_MAX_TURNS",
_memory_defaults.short_term_max_turns
        ),

summary_every_n_turns=_get_int(

"NEXUS_SUMMARY_EVERY_N_TURNS",
_memory_defaults.summary_every_n_turn
s
        ),
    )

    security = SecurityConfig(
        audit_log_path=_get(
            "NEXUS_AUDIT_LOG_PATH",
```

```
_security_defaults.audit_log_path
        ),

encrypt_memory_at_rest=_get_bool(

"NEXUS_ENCRYPT_MEMORY_AT_REST",
_security_defaults.encrypt_memory_at_
rest
        ),

max_actions_per_minute=_get_int(

"NEXUS_MAX_ACTIONS_PER_MINUTE",
_security_defaults.max_actions_per_mi
nute
        ),
    )

    config = NexusConfig(
        environment=environment,

log_level=_get("NEXUS_LOG_LEVEL",
"INFO"),
        log_dir=_get("NEXUS_LOG_DIR",
"./data/logs"),

data_dir=_get("NEXUS_DATA_DIR",
"./data"),
        voice=voice,
```

```
        llm=llm,
        memory=memory,
        security=security,
    )

    if overrides:
        config =
_apply_overrides(config, overrides)

    _validate(config)
    return config


def _apply_overrides(config:
NexusConfig, overrides: dict[str,
Any]) -> NexusConfig:
    """
    Apply dotted-key overrides (e.g.
``{"llm.model_name": "gpt-5"}``) to a
    frozen ``NexusConfig`` by
rebuilding the affected nested
dataclasses.
    Used mainly by tests that need
one or two fields tweaked without
    reimplementing the full env-var
precedence chain.
    """
    import dataclasses
```

```
    top_level_updates: dict[str, Any]
= {}
    nested_updates: dict[str,
dict[str, Any]] = {}

    for key, value in
overrides.items():
        if "." in key:
            section, sub_key =
key.split(".", 1)

nested_updates.setdefault(section,
{})[sub_key] = value
        else:
            top_level_updates[key] =
value

    for section, updates in
nested_updates.items():
        if not hasattr(config,
section):
            raise
ConfigurationError(f"Unknown config
section in override: {section!r}")
        current_section =
getattr(config, section)
        new_section =
dataclasses.replace(current_section,
**updates)
```

```
        top_level_updates[section] =
new_section

    return
dataclasses.replace(config,
**top_level_updates)


def _validate(config: NexusConfig) ->
None:
    """
    Cross-field validation that can't
be expressed as a single field's
    default. Raises
``ConfigurationError`` with a clear,
actionable message
    on the first failure found.
    """
    if config.environment ==
Environment.PRODUCTION and not
config.llm.api_key:
        if config.llm.provider !=
"ollama":  # local models need no API
key
            raise ConfigurationError(
                "Production
environment requires
NEXUS_LLM_API_KEY to be set "
                f"for provider
```

```
{config.llm.provider!r}."
            )

    if not (0.0 <=
config.voice.wake_word_sensitivity <=
1.0):
        raise ConfigurationError(

"NEXUS_WAKE_WORD_SENSITIVITY must be
between 0.0 and 1.0, got "
            f"
{config.voice.wake_word_sensitivity}"
        )

    if not (0.0 <=
config.voice.vad_aggressiveness <=
1.0):
        raise ConfigurationError(
            "NEXUS_VAD_AGGRESSIVENESS
must be between 0.0 and 1.0, got "
            f"
{config.voice.vad_aggressiveness}"
        )

    if
config.security.max_actions_per_minut
e <= 0:
        raise
ConfigurationError("NEXUS_MAX_ACTIONS
```

```
_PER_MINUTE must be positive.")

    if config.llm.max_tokens <= 0:
        raise
ConfigurationError("NEXUS_LLM_MAX_TOK
ENS must be positive.")
```